DAY 2

```
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;

int main() {

    // ======================================================
    // DOUBT 1: INPUT A FULL SENTENCE USING getline()
    // ======================================================
    cout << "===== DOUBT 1: FULL SENTENCE =====\n";

    string sentence;

    cout << "Enter a complete sentence: ";
    getline(cin, sentence);

    cout << "Your sentence: " << sentence << "\n\n";


    // ======================================================
    // DOUBT 2: CALCULATE x TO THE POWER y USING pow()
    // ======================================================
    cout << "===== DOUBT 2: POWER =====\n";

    int x, y;

    cout << "Enter base x and power y: ";
    cin >> x >> y;

    cout << x << "^" << y << " = " << pow(x, y) << "\n\n";


    // ======================================================
    // 1. ARRAYS
    // ======================================================
    cout << "===== 1. ARRAYS =====\n";

    int a[] = {1, 2, 3};
    int b[5] = {0, 10, 11};
    int c[5] = {0};

    cout << "Array a: ";
    for (int i = 0; i < 3; i++)
        cout << a[i] << " ";

    cout << "\nArray b: ";
    for (int i = 0; i < 5; i++)
        cout << b[i] << " ";
```

```
cout << "\nArray c: ";
for (int i = 0; i < 5; i++)
    cout << c[i] << " ";

cout << "\n\n";


// ======================================================
// 2. DYNAMIC ARRAYS USING POINTERS
// ======================================================
cout << "===== 2. DYNAMIC ARRAYS =====\n";

int size;

cout << "Enter dynamic array size: ";
cin >> size;

if (size <= 0) {
    cout << "Invalid size.\n\n";
}
else {
    int* ptr = new int[size];

    cout << "Enter " << size << " elements: ";

    for (int i = 0; i < size; i++)
        cin >> ptr[i];

    cout << "Dynamic array: ";
    for (int i = 0; i < size; i++)
        cout << ptr[i] << " ";

    cout << "\nFirst element: " << *ptr;
    cout << "\nBase address: " << ptr;

    if (size > 2)
        cout << "\nAddress of index 2: " << &ptr[2];

    delete[] ptr;
    ptr = nullptr;

    cout << "\nMemory released.\n\n";
}


// ======================================================
// 3. VECTORS AND IMPORTANT METHODS
// ======================================================
cout << "===== 3. VECTORS =====\n";

vector<int> v = {10, 20, 30};

// Add an element at the end
v.push_back(40);
```

```
cout << "After push_back(): ";
for (int num : v)
    cout << num << " ";

// Remove the last element
v.pop_back();

cout << "\nAfter pop_back(): ";
for (int num : v)
    cout << num << " ";

// Insert 15 at index 1
v.insert(v.begin() + 1, 15);

cout << "\nAfter insert(): ";
for (int num : v)
    cout << num << " ";

// Remove the element at index 1
v.erase(v.begin() + 1);

cout << "\nAfter erase(): ";
for (int num : v)
    cout << num << " ";

cout << "\nVector size: " << v.size() << "\n\n";


// ======================================================
// 4. BINARY SEARCH
// Array must be sorted
// Time Complexity: O(log n)
// ======================================================
cout << "===== 4. BINARY SEARCH =====\n";

vector<int> arr = {-1, 0, 3, 5, 9, 12};

int target;

cout << "Enter target to search: ";
cin >> target;

int low = 0;
int high = arr.size() - 1;
int index = -1;

while (low <= high) {
    int mid = low + (high - low) / 2;

    cout << "Checking: " << arr[mid] << "\n";

    if (arr[mid] == target) {
        index = mid;
        break;
    }
```

```
        else if (arr[mid] < target) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }

    if (index != -1)
        cout << "Target found at index: " << index << "\n\n";
    else
        cout << "Target not found.\n\n";


    // ======================================================
    // 5. ROTATE ARRAY TO THE RIGHT BY K POSITIONS
    // ======================================================
    cout << "===== 5. ARRAY ROTATION =====\n";

    vector<int> rot = {1, 2, 3, 4, 5};

    int k;

    cout << "Enter number of right rotations: ";
    cin >> k;

    if (!rot.empty() && k >= 0) {
        k = k % rot.size();

        for (int i = 0; i < k; i++) {
            // Store the last element
            int temp = rot.back();

            // Remove the last element
            rot.pop_back();

            // Insert it at the beginning
            rot.insert(rot.begin(), temp);
        }
    }

    cout << "Array after rotation: ";

    for (int num : rot)
        cout << num << " ";

    cout << "\n";

    return 0;
}
```

NOTES

⇒ Arrays → collection of same type of objects

* int a[] = {1,2,3};
  int a[size]; a[] = {0,10,11};
  int a[size] = {0};

* Array in heap using pointers
  int * a = new int[size]; → memory allocated
  Memory deallocation is necessary in this case.
  delete a[];

* The array identifier is the pointer to the first element in the array.
  Name of array refers to the base address of the array.
  address of arr[i] = Base address + (i × size of data type)

* While solving array questions keep in mind 2 boundary conditions:
  ① If array is empty
  ② If array is already full

* Array has fixed size. To add an element to an full array:
  → declare another array of size +1.
  → copy the elements of the original array
  → add the new element
  → deallocate the memory of the previous array

deallocate the memory of the previous ...

* Vectors → flexible size array
  ↳ in #include<vector>

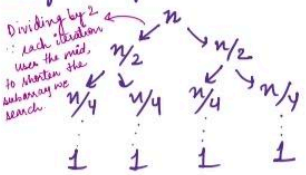
Important vector methods:
1. v.push_back(5)
2. v.pop_back()
3. v.insert()
4. v.erase()
5. v.size()

⇒ Binary Search
- divide the array into parts and search accordingly.
- It is a divide and conquer method

eg:- a = [-1, 0, 3, 5, 9, 12]
target = 9
in every iteration check the middle element.

Time Complexity of Binary search:

Size of array = n.



Let search be done till $\frac{n}{2^k} = 1$ (i.e. size of subarray = 1 we are searching)

$\therefore n = 2^k \Rightarrow k = \log_2 n$

$\therefore$ TC of BS = $O(\log_2 n)$

General format for Binary Search:

```
int low = 0;
int high = max.value;
while (low <= high)
  { // conditions }
return;
```

* Rotating an array

a → [1,2,3,4,5], k=1 → no. of rotations

5,1,2,3,4 → ans

k=2, a → [1,2,3,4,5]
↓ 1 rot
5,1,2,3,4
↓ 2 rot
4,5,1,2,3

```
func(vector a, int k)
{ for (int i=1; i<=k; i++)
  { int temp = a[a.size()-1];
    a.pop_back();
    a.insert(a.begin(), temp);
  }
}
```